# Lesson : ORM vs. Raw Queries

## Introduction

Object-Relational Mapping (ORM) is a powerful technique that bridges the gap between object-oriented programming and relational databases, simplifying database interactions by allowing developers to work with familiar programming constructs instead of raw SQL. This lesson explores ORM concepts, their advantages, and limitations, with a focus on Prisma—a modern ORM tool tailored for PostgreSQL. You will learn how to integrate Prisma into an Express.js project, define database schemas, apply migrations, and perform CRUD operations efficiently.

## Learning Outcomes

By the end of this lesson, you will be able to:

1. Define ORM, explain its purpose, and identify its benefits and limitations.
2. Initialize a project, configure Prisma, and connect it to a PostgreSQL database using environment variables.
3. Create and manage database models using Prisma's schema-first approach.
4. Generate and apply database migrations to keep your schema in sync with your application.
5. Use Prisma Client to create, read, update, and delete records in an Express.js application.
6. Utilize Prisma's type-safe queries, auto-generated migrations, and real-time subscriptions to enhance developer productivity and application performance.

---

## Object Relational Mapping

Object Relational Mapping (ORM) is a technique that bridges the gap between object-oriented programming languages and relational databases. In simpler terms, it allows developers to interact with databases using the same syntax and structure as their chosen programming language, eliminating the need to write raw SQL queries. This makes database operations easier, improves code readability, and enhances maintainability.

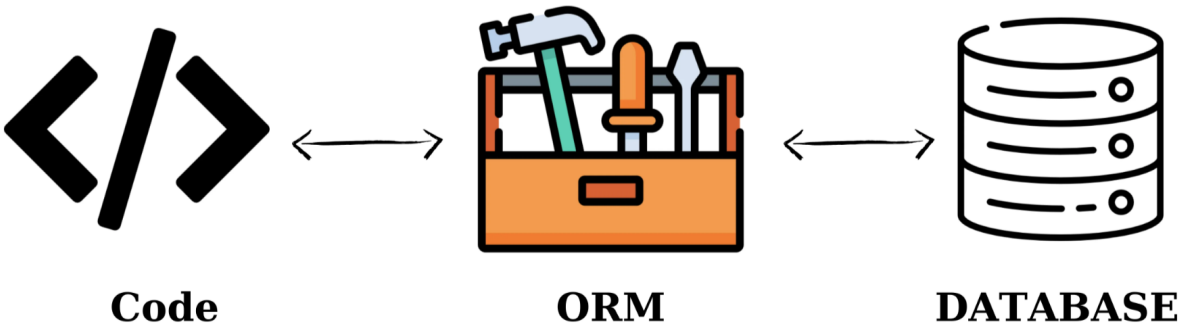

For example, instead of writing a query like `SELECT * FROM users` to retrieve data from a database, you can use something like `User.find()` in your code. Behind the scenes, the ORM framework translates this object-oriented call into the corresponding SQL query. This abstraction simplifies how developers work with databases by letting them focus on the logic of their application rather than the intricacies of SQL.

---

## ORM Methodology

ORM operates by mapping objects in your code to tables in a relational database. Here's how this works:

- A class in your code represents a table.
- Instances of the class represent rows in the table.
- Attributes (or properties) of the class correspond to columns in the table.




This mapping hides the complexity of SQL queries and encapsulates them into methods or functions. Instead of manually writing SQL, you work with objects and methods that represent database operations. For instance, consider the following JavaScript code:

```
const users = await User.findAll({ where: { age: { $gt: 18 } } });
```

Here, `User.findAll()` fetches all users older than 18. The ORM framework automatically generates the equivalent SQL query (`SELECT * FROM users WHERE age > 18`) for you. This means you don't need to worry about crafting SQL statements manually, which saves time and reduces errors.

## Advantages of Using ORM

ORM frameworks offer several benefits that make them a valuable tool for developers:

**Simplified Syntax**

ORMs allow you to write database queries in the same language as your application. For example, instead of writing:

```
SELECT * FROM posts WHERE user_id = 1;
```

You can use:

```
const posts = await Post.findAll({ where: { userId: 1 } });
```

This approach feels more natural because it aligns with the programming language you're already using. It also makes the code easier to read and maintain.

**Modularity of Database Logic** By abstracting SQL queries, ORMs keep your application logic clean and organized. This separation ensures that your codebase remains easier to debug and maintain. For instance, if you later decide to change how data is retrieved, you only need to update the ORM methods without touching multiple parts of your code.

**Efficiency for Complex Projects** In large-scale applications with numerous tables and relationships, ORMs reduce boilerplate code. Defining relationships between tables becomes straightforward. For example:

```
class User extends Model {}
class Post extends Model {}
User.hasMany(Post); // A user can have many posts
Post.belongsTo(User); // Each post belongs to a user
```

These lines define relationships between the `User` and `Post` tables. The ORM handles foreign keys and ensures consistency automatically, so you don't need to write complex SQL joins or manage relationships manually.

**Database Flexibility**

Most ORMs allow you to switch between different database systems (e.g., MySQL, PostgreSQL, SQLite) without rewriting your queries. This flexibility is invaluable when scaling or migrating

applications. For example, if you start with SQLite during development but later switch to PostgreSQL for production, the ORM will handle the transition smoothly.

## Limitations of ORM

While ORMs offer significant advantages, they also have some limitations that developers should be aware of:

1. **Data Inconsistency**: If you delete an entity in your code, the corresponding table in the database won't be deleted automatically. Similarly, renaming an entity can create a new table, leaving the old one untouched and causing inconsistencies. To avoid this, developers need to carefully manage schema changes.
2. **Performance Overhead**: ORM-generated queries may not always be as optimized as manually written SQL queries. This can lead to performance issues in scenarios requiring highly efficient database operations. For critical queries, developers might still need to write raw SQL to ensure optimal performance.
3. **Limited Data Types**: Some ORMs have limited support for advanced database types, which may require you to store data in a less flexible way. For example, if your database supports JSON fields but your ORM doesn't fully support them, you might need to find workarounds.

## Practical Examples of ORM in Action

To illustrate how ORM simplifies database interactions, let's explore a few practical examples.

**Example: Querying Data**

Without ORM:

```
SELECT * FROM users WHERE age > 18;
```

With ORM (using Sequelize in JavaScript):

```
const users = await User.findAll({
  where: {
    age: { [Op.gt]: 18 } // Op.gt means "greater than"
  }
});
console.log(users);
```

**Explanation**: The `User.findAll()` method retrieves all records from the `users` table where the age is greater than 18. The ORM translates this into the equivalent SQL query, making it easier to work with databases without writing raw SQL.

**Example: Defining Relationships**

Without ORM:

```
CREATE TABLE users (
  id INT PRIMARY KEY,
  name VARCHAR(255)
);
CREATE TABLE posts (
  id INT PRIMARY KEY,
  user_id INT,
  title VARCHAR(255),
  FOREIGN KEY (user_id) REFERENCES users(id)
);
```

With ORM (using Sequelize):

```
class User extends Model {}
class Post extends Model {}
User.hasMany(Post); // A user can have many posts
Post.belongsTo(User); // Each post belongs to a user
```

**Explanation**:

The `hasMany` and `belongsTo` methods define relationships between tables. The ORM handles foreign keys and ensures consistency automatically, so you don't need to write complex SQL statements to manage relationships.

**Example: Inserting Data**

Without ORM:

```
INSERT INTO users (name, age) VALUES ('Alice', 25);
```

With ORM (using Sequelize):

```
await User.create({ name: 'Alice', age: 25 });
```

**Explanation**: The `User.create()` method inserts a new record into the `users` table. The ORM handles escaping and validation automatically, reducing the risk of SQL injection and ensuring data integrity.

## Popular ORM Frameworks

Different programming languages have their own ORM tools tailored to their ecosystems. Below are some widely used ORM frameworks:

- **C#**: Entity Framework, Dapper
- **Java**: Hibernate, JPA
- **Python**: Django ORM, SQLAlchemy

- **JavaScript**: Sequelize, Mongoose (for MongoDB), TypeORM

Each of these frameworks provides similar functionality but is designed to work seamlessly with its respective language and ecosystem.

---

## Prisma as an ORM Tool for Postgres


Prisma distinguishes itself from other ORM tools through its unique architecture and developer-friendly features:

**Type-Safe Queries** Prisma generates a type-safe client based on your database schema. This ensures that queries are validated at compile time, reducing runtime errors and improving code reliability. For example:

```
const users = await prisma.user.findMany({
  where: { age: { gt: 18 } },
});
```

The `gt` operator (greater than) is type-checked, ensuring that only valid comparisons are made.

**Schema-First Approach** Prisma uses a declarative schema file (`schema.prisma`) to define your database structure. This file serves as the single source of truth for both your application and database. For example:

```
model User {
  id    Int    @id @default(autoincrement())
  name  String
  age   Int
  posts Post[]
}
model Post {
  id       Int    @id @default(autoincrement())
  title    String
  userId   Int
  user     User   @relation(fields: [userId], references: [id])
}
```

Here, the `User` and `Post` models define tables with their relationships. Prisma automatically generates the necessary SQL migrations to sync this schema with your PostgreSQL database.

**Auto-Generated Migrations** Prisma simplifies database migrations by generating SQL scripts based on changes in the `schema.prisma` file. Running a migration is as simple as:

```
npx prisma migrate dev --name init
```

This command applies the schema changes to your database while keeping a versioned history of migrations.

**Real-Time Data Updates** Prisma supports **real-time subscriptions**, enabling your application to listen for changes in the database. For example, you can subscribe to new records being added to a table:

```
const newUserSubscription = await prisma.user.subscribe({
  where: { mutation: 'CREATED' },
});
newUserSubscription.on('data', (user) => {
  console.log('New user created:', user);
});
```

## Features of Prisma for Postgres

Prisma is particularly well-suited for PostgreSQL due to its advanced support for relational databases. Below are some key features that make it an excellent choice:

**Support for PostgreSQL-Specific Features** Prisma fully supports PostgreSQL-specific features like JSONB, arrays, and full-text search. For example, you can store and query JSON data seamlessly:

```
model Profile {
  id     Int    @id @default(autoincrement())
  userId Int
  data   Json
  user   User   @relation(fields: [userId], references: [id])
}
```

Querying JSON fields is straightforward:

```
const profiles = await prisma.profile.findMany({
  where: {
    data: {
      path: ['preferences', 'notifications'],
      equals: 'enabled',
    },
  },
});
```

**Efficient Query Execution** Prisma optimizes queries to minimize performance overhead. It generates efficient SQL queries based on your Prisma Client calls, ensuring fast execution even for complex operations.

**Integration with Modern Frameworks** Prisma integrates seamlessly with popular frameworks like **Next.js**, **Express**, and **NestJS**, making it a versatile choice for modern applications. For example, in a Next.js API route:

```
import { prisma } from '../../../lib/prisma';
export default async function handler(req, res) {
  const users = await prisma.user.findMany();
  res.status(200).json(users);
}
```

---

## Practical Example: Using Prisma with PostgreSQL

To demonstrate how Prisma works, let's walk through a simple example of setting up and querying a PostgreSQL database.

## Define the Schema

Create a `schema.prisma` file:

```
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}
model User {
  id    Int    @id @default(autoincrement())
  name  String
  email String @unique
  posts Post[]
}
model Post {
  id       Int    @id @default(autoincrement())
  title    String
  content  String?
  authorId Int
  author   User   @relation(fields: [authorId], references: [id])
}
```

## Generate and Apply Migrations

Run the following commands to create and apply migrations:

```
npx prisma migrate dev --name init
```

## Query the Database

Use Prisma Client to interact with the database:

```
import { PrismaClient } from '@prisma/client';
const prisma = new PrismaClient();
async function main() {
  // Create a new user
  const user = await prisma.user.create({
    data: {
      name: 'Alice',
      email: 'alice@example.com',
      posts: {
        create: { title: 'My First Post', content: 'Hello, world!' },
      },
    },
  });
  console.log('Created user:', user);
  // Fetch all users with their posts
  const usersWithPosts = await prisma.user.findMany({
    include: { posts: true },
  });
  console.log('Users with posts:', usersWithPosts);
}
main()
  .catch((e) => console.error(e))
  .finally(async () => await prisma.$disconnect());
```

## Advantages of Using Prisma with PostgreSQL

1. **Ease of Use**: Prisma's intuitive API and schema-first approach make it easy to get started, even for beginners.
2. **Performance Optimization**: Prisma generates efficient SQL queries, ensuring minimal performance overhead.
3. **Developer Productivity**: Features like auto-generated migrations and real-time subscriptions boost productivity and reduce boilerplate code.
4. **Community and Ecosystem**: Prisma has a growing community and extensive documentation, making it easier to find solutions to common challenges.

---

## Setting Up Prisma in an Express.js Project

Integrating Prisma into an **Express.js** project streamlines database interactions by providing a type-safe and developer-friendly ORM layer. This setup allows you to build scalable APIs with minimal code while leveraging the power of PostgreSQL or other relational databases.

## Step 1: Initialize the Express.js Project

Before adding Prisma, ensure you have a basic Express.js project set up. If you're starting from scratch, follow these steps:

#### Create a New Project Directory

```
mkdir express-prisma-app
cd express-prisma-app
```

**Initialize the Project** Use `npm` to initialize the project and install Express:

```
npm init -y
npm install express
```

**Set Up the Basic Server** Create an `index.js` file for your Express server:

```
const express = require('express');
const app = express();
const PORT = 3000;
app.use(express.json());
app.get('/', (req, res) => {
  res.send('Hello, Express with Prisma!');
});
app.listen(PORT, () => {
  console.log(`Server is running on http://localhost:${PORT}`);
});
```

**Test the Server** Run the server using:

```
node index.js
```

Visit `http://localhost:3000` in your browser to confirm the server is running.

## Step 2: Install and Configure Prisma

Once your Express.js project is ready, you can integrate Prisma as follows:

**Install Prisma CLI and Client** Install the necessary Prisma packages:

```
npm install @prisma/client
npm install prisma --save-dev
```

**Initialize Prisma** Run the following command to create a Prisma directory and configuration file:

```
npx prisma init
```

This generates a `prisma` folder containing:

- `schema.prisma`: The schema file where you define your database models.
- `.env`: A file to store environment variables, including your database connection string.

**Configure the Database Connection** Open the `.env` file and update the `DATABASE_URL` with your PostgreSQL connection string:

```
DATABASE_URL="postgresql://USER:PASSWORD@HOST:PORT/DATABASE?schema=public"
```

Replace `USER`, `PASSWORD`, `HOST`, `PORT`, and `DATABASE` with your actual database credentials.

## Step 3: Define the Database Schema

With Prisma initialized, you can define your database schema in the `schema.prisma` file. For example, let's create two models: `User` and `Post`.

```
model User {
  id    Int    @id @default(autoincrement())
  name  String
  email String @unique
  posts Post[]
}
model Post {
  id       Int    @id @default(autoincrement())
  title    String
  content  String?
  authorId Int
  author   User   @relation(fields: [authorId], references: [id])
}
```

This schema defines:

- A `User` model with fields for `id`, `name`, `email`, and a relationship to `Post`.
- A `Post` model with fields for `id`, `title`, `content`, and a foreign key linking it to a `User`.

## Step 4: Apply Migrations

After defining the schema, apply the changes to your database using Prisma Migrate:

**Generate and Apply Migrations**
Run the following command:

```
npx prisma migrate dev --name init
```

This creates and applies the initial migration, syncing your database schema with the `schema.prisma` file.

**Verify the Migration** Check your database to confirm that the `User` and `Post` tables have been created.

## Step 5: Integrate Prisma with Express.js

Now that Prisma is set up, you can use it to interact with your database in your Express routes.

**Generate Prisma Client** Generate the Prisma Client based on your schema:

```
npx prisma generate
```

**Set Up Prisma in Your Application** Import and initialize Prisma Client in your `index.js` file:

```
const { PrismaClient } = require('@prisma/client');
const prisma = new PrismaClient();
// Middleware to handle Prisma disconnection on server shutdown
const closePrisma = async () => {
  await prisma.$disconnect();
};
process.on('SIGINT', closePrisma);
process.on('SIGTERM', closePrisma);
```

**Create API Endpoints** Add routes to perform CRUD operations. For example:

```
// Create a new user
app.post('/users', async (req, res) => {
  const { name, email } = req.body;
  const user = await prisma.user.create({
    data: { name, email },
  });
  res.json(user);
});

// Fetch all users
app.get('/users', async (req, res) => {
  const users = await prisma.user.findMany({
    include: { posts: true }, // Include related posts
  });
  res.json(users);
});

// Create a new post for a user
app.post('/posts', async (req, res) => {
  const { title, content, authorId } = req.body;
  const post = await prisma.post.create({
    data: { title, content, authorId },
  });
  res.json(post);
});
```

## Step 6: Test the API

Start your server and test the endpoints using tools like **Postman** or **cURL**:

**Create a User** Send a `POST` request to `/users` with the following JSON body:

```
{
  "name": "Alice",
  "email": "alice@example.com"
}
```

**Fetch All Users**

Send a `GET` request to `/users` to retrieve all users and their associated posts.

**Create a Post for a User**
Send a `POST` request to `/posts` with the following JSON body:

```
{
  "title": "My First Post",
  "content": "Hello, world!",
  "authorId": 1
}
```

---

## Conclusion

ORM tools like Prisma simplify database interactions, making it easier to build and maintain applications without writing complex SQL queries. By integrating Prisma into an Express.js project, you can leverage its intuitive API, type-safe queries, and seamless PostgreSQL support to streamline development and improve efficiency. With features like auto-generated migrations and real-time data updates, Prisma not only enhances developer productivity but also ensures scalability and reliability.